Reviewer Pack — Minimal Local Induction Dimension and Communication Complexi…

Entry f5e2637983f8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 22:17:05 UTC

Minimal Local Induction Dimension and Communication Complexity Rank

Entry ID: `f5e2637983f8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 22:17:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Induction Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given n-ary Boolean function f, the communication complexity rank of the associated partial commutative matrix is Θ(log^2(n) * lnd(f)), where lnd(f) denotes the local induction dimension of f.

Rationale (proposer's reasoning):

> Local Induction Dimension measures the combinatorial complexity of a set and can be used to represent the structure of Boolean functions. This conjecture posits that this algebraic topology invariant has a direct relationship with communication complexity, which is a measure of the amount of communication needed for two parties to compute a function together.

Taxonomy category: `AlgebraicToplogy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `28d5ed822e472815`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given n-ary Boolean function f with lnd(f) ≤ 40, if the ratio of the rank of the associated partial commutative matrix M_f to log^2(n) * lnd(f) is within ±10% of 1 for all 30 seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `lnd(f) AND communication complexity rank` - `partial commutative matrix AND local induction dimension Θ(log^2(n))` - `algebraic topology AND communication complexity matrix rank`

Top relevant hits considered: - [s2:10.1145/3654780.3654788] Guest Column: Structure in Communication Complexity and Constant-Cost Complexity Classes - [s2:2401.14623] Structure in Communication Complexity and Constant-Cost Complexity Classes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def local_induction_dimension(f):
        n = int(math.log2(len(f)))
        count = 0
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    count += 1
        return count

    def partial_commutative_matrix(f):
        n = int(math.log2(len(f)))
        M = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    M[i][j] = 1
                    M[j][i] = 1
        return M

    def matrix_rank(M):
        m, n = len(M), len(M[0])
```

```
        rank = 0
        for i in range(m):
            if all(M[i][j] == 0 for j in range(n)):
                continue
            pivot_col = next(j for j in range(n) if M[i][j] != 0)
            for j in range(n):
                if j != pivot_col:
                    factor = M[j][pivot_col] / M[i][pivot_col]
                    for k in range(m):
                        M[k][j] -= factor * M[k][pivot_col]
            rank += 1
        return rank

    def log2(x):
        return math.log(x, 2)

    n = random.randint(5, 30)
    f = generate_boolean_function(n)
    lnd_f = local_induction_dimension(f)
    if lnd_f > 40:
        return {
            "metric_name": "communication_complexity_rank",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "lnd(f) > 40"
        }

    M_f = partial_commutative_matrix(f)
    rank_M_f = matrix_rank(M_f)

    ratio = rank_M_f / (log2(n)**2 * lnd_f)
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(ratio - 1) <= 0.1,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
```

```
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

`TIMEOUT after 240s`

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support conditions could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12097
2	propose	ollama_remote	glm4:latest	0	0	12444
3	preregistration	ollama_remote	glm4:latest	0	0	9158
4	novelty	ollama_remote	glm4:latest	0	0	8461
5	novelty	ollama_remote	glm4:latest	0	0	9901
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20466
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15192
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10950
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9815
10	judge	ollama_remote	glm4:latest	0	0	18842

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 127325 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f5e2637983f8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f5e2637983f8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f5e2637983f8.tar.gz` (if generated)